Ex. No. 12	GUI development with database integration
Date of Exercise	**07-04-2026**
Git-hub Host Link	https://github.com/chrisharrison-byte/GUI-development-with-database-integration/tree/main

Aim

To develop a GUI-based desktop application integrated with a database using Java Swing and JDBC to perform data insertion and retrieval.

Description:

This experiment combines Java Swing (GUI) and JDBC (Database Connectivity) to build a real-time application.

- Swing is used to design the graphical interface.
- JDBC is used to connect and interact with the database.

The application allows users to:

- Enter student details through a GUI
- Store data in a database
- Retrieve and display stored data

This demonstrates:

- Event handling in GUI
- Database connectivity
- Executing SQL queries
- Real-world application development

Procedure:

Step 1 : Start the program.

Step 2 : Import the required packages: javax.swing, java.awt.event, and java.sql.

Step 3 : Create a class that extends JFrame.

Step 4 : Add GUI components such as labels, text fields, and buttons.

Step 5 : Set layout and add components to the frame.

Step 6 : Establish database connection using JDBC.

Step 7 : Write SQL query to insert data into the database.

Step 8 : Handle button click event using ActionListener.

Step 9 : On button click, read input from text fields and store it in the database.

Step 10 : Display success message after insertion.

Step 11 : Close the database connection.

Step 12 : Stop the program.

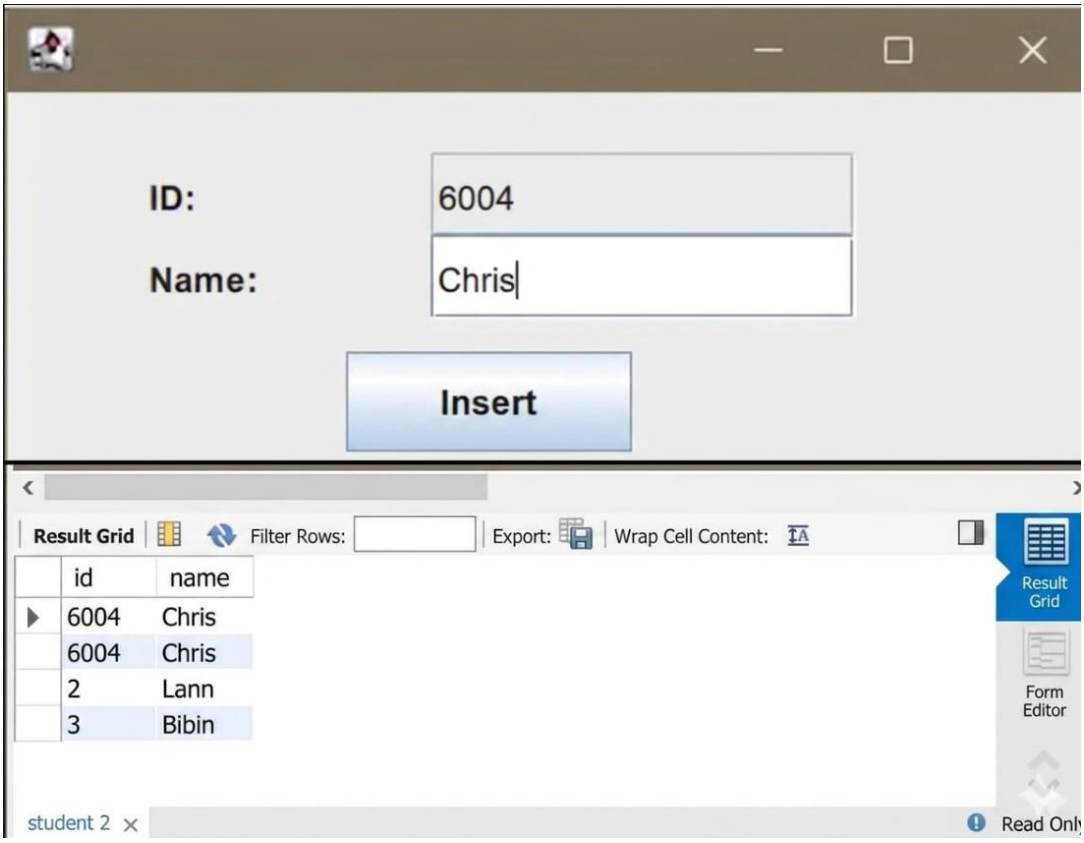
Program:

```
import javax.swing.*;
import java.awt.event.*;
import java.sql.*;

public class SwingJDBCDemo extends JFrame implements ActionListener {

    JLabel l1, l2;
    JTextField t1, t2;
    JButton b1;

    Connection con;
    PreparedStatement ps;

    SwingJDBCDemo() {
```

```
    // GUI Components
    l1 = new JLabel("ID:");
    l1.setBounds(50, 50, 100, 30);

    t1 = new JTextField();
    t1.setBounds(150, 50, 150, 30);

    l2 = new JLabel("Name:");
    l2.setBounds(50, 100, 100, 30);

    t2 = new JTextField();
    t2.setBounds(150, 100, 150, 30);

    b1 = new JButton("Insert");
    b1.setBounds(120, 150, 100, 30);

    b1.addActionListener(this);

    add(l1); add(t1);
    add(l2); add(t2);
    add(b1);

    setSize(400, 300);
    setLayout(null);
    setVisible(true);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    // Database Connection
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
        con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/studentdb", "root", "password");
    } catch (Exception e) {
        System.out.println(e);
    }
}

public void actionPerformed(ActionEvent e) {
    try {
        String id = t1.getText();
```

```
            String name = t2.getText();

            String query = "INSERT INTO student VALUES (?, ?)";
            ps = con.prepareStatement(query);

            ps.setInt(1, Integer.parseInt(id));
            ps.setString(2, name);

            ps.executeUpdate();

            JOptionPane.showMessageDialog(this, "Record Inserted Successfully");

        } catch (Exception ex) {
            System.out.println(ex);
        }
    }

    public static void main(String[] args) {
        new SwingJDBCDemo();
    }
}
```

Output



Result

Thus, a GUI-based application integrated with a database was successfully developed using Java Swing and JDBC for inserting records.